Lab #5
EGRE246

1 Overview

Remember that in C a string is an array of type `char` with a null character (ASCII 0, denoted by `'\0'` in C). C strings have been rightly criticized because you never know how long it is until you scan for and find this ending null character. A more reasonable way to define a string is to define it as a structure where you keep track of the string's current length at all times (and do not therefore need to mark the end of the string with a null).

For this lab you are to define your own string data type as defined below. You will need three files – `string.h` (the specification), `string.c` (the implementation), plus a driver program (let's call it `driver.c`) (or, if you choose, a Unity test file) to exercise and test your string routines. You are not allowed to use any of the C string manipulation routines as defined in the C standard string library. **In addition your strings cannot and should not be terminated by the null character as standard strings are in C (i.e. you will need to use a different method to keep track of the end of your strings, as mentioned above)**.

2 File `string.h`

This file is the specification file for your type, i.e. it will include your type definition for `string` and all of your function prototypes. It is downloadable off of the Canvas class pages. **You will need to edit this file.**

Your `string` type will be a pointer to a structure called `stringType`. Define the following preprocessor constant and type (i.e. add this to the file):

```
#define MAX_STRING_SIZE 128 // arbitrary
typedef struct stringType *string;
```

Now add the appropriate fields to `stringType`.

```
struct stringType {
  // your definitions go here (you will only need 2 fields - one
  // to keep track of the length, the other a char array of
  // MAX_STRING_SIZE length. To make things easier just define your
  // data field as a char array in your structure - you will not
  // need to malloc it in screate at runtime!)
};
```

What then is a variable of type `string`? Note that `string` has been defined to be a pointer type which means that when you define

```
string str;
```

you do not need to put an * in front of str when you declare it as the typedef for string (see above) already has define it as a pointer. So with the variable str is a pointer to structure stringType and you would access stringType's fields with str->[your field name here]. Note again that if you keep track of the length of a string you will always know where it ends and you no longer need to mark the end of a string with a '\0'. **Therefore do not mark the end of your strings with '\0' and none of your code (except for scopy) should every check for it or use it!**

All of the prototypes for the functions you will define to manipulate strings are in string.h file. For this lab you will define the following (which are described in the next section):

```
string screate();  // returns a newly created string
void scopy(string s,char *c);// copies a null-terminated C string c to s
int  slen(string); // returns the length of a string
void printString(string); // prints a string to standard output w/o newline
```

3 File string.c

File string.c is the implementation file for your new string type and all of your functions; the beginnings of it can be found on Canvas. You will begin this file by including your specification file string.h; later you might add other header file includes if they are needed.

Now you define the following:

- string screate();
  Use malloc to return a pointer to structure stringType. Note that to get the size of your structure you need to use sizeof(struct stringType), not sizeof(string). (Why?) Also remember that your data field will be an array of type char of MAX_STRING_SIZE size.

- printString(string)
  I would write printString next so you will have a means to inspect your strings in debugging. Output nothing if the string is empty. Here's most of it:

  ```
  void printString(string s) {
    for(int i = 0; i < slen(s); i++)
      printf("%c",s->/* your data field name here */);
  }
  ```

- void scopy(string s,char *cstr);
  This will copy all of the characters from the null-terminated C string cstr to your string s. Remember that legal C strings are always terminated by the null character '\0'; therefore you should set up a loop that copies all of the characters from cstr to the data field in s until you reach the end of cstr. However, do not copy over this null character - you are using another way to keep track of the end of your strings so be sure to update the appropriate field.

  ```
  void scopy(string s,char *cstr) {
  ```

```
  // your stuff here, as needed
  int i = 0;
  while(cstr[i] != '\0' && i < MAX_STRING_SIZE) {
    // -- copy the i-th char in cstr to the
    //    i-th location in the data field in s
    // -- increment the size field in s
  }
}
```

Also don't forget that since your strings are a pointer to a structure you can use the arrow (`->`) operator to access fields in `stringType` (e.g. if you had field named `kraken` of type float in `stringType`, you could access that field by '`s->kraken = 0.0`'. Also it's OK if you just assume that the string `s` is big enough to copy over `cstr` (in other words, the function is considered undefined if it's not big enough).

- `int slen(string s);`
  Note that this function is very easy to write if you define and keep track of the size of your string in structure `stringType`.

4 File `driver.c`

Your driver program should test all of the constructs you define above. Here is an example starting point (note that it does not *completely* test your code):

```
#include <stdio.h>
#include "string.h"

int main(void){
  string s1 = screate();
  scopy(s1,"hi mom");
  printf("s1 length = %d\n",slen(s1));
  printString(s1); printf("\n");
  return 0;
}
```

As we covered in class remember that you will compile your project with the following line:

```
gcc -Wall string.c driver.c
```

Or, if you are using onlineGDB, you will need to have all of the files *including the header file* as a file (a tab) in your onlineGDB page.